End-to-End Data Management Pipeline for a Recommendation System
DM4ML Assignment 2

Git link: https://github.com/2025ag05075-Surya/recommendation-pipeline

1. Overview and design decisions

The pipeline runs seven stages – ingestion, validation, preparation, transformation, feature management, training and evaluation – inside one Prefect flow (architecture diagram: results/Architecture.png)

Design principles:

(i) Modular code with one package per stage and a single configs/config.yaml

(ii) Reproducible – all randomness is seeded and two full runs give identical result files

(iii) No leakage – the split is by time and all derived datasets and features are computed from the training period only.

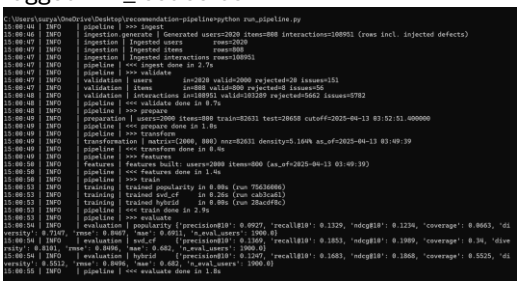
(iv) Features defined once in a documented catalog and served from a single store.

Data: MovieLens-style synthetic dataset is generated (2,000 users, 800 items, 108,951 raw interactions, ratings 1–5, 12 genres, 1–3 per item)

2. Ingestion, validation, preparation, transformation

Ingestion reads every column as text, Type problems are detected by validation instead of being silently coerced. Validation implements rules for missing values, duplicates, wrong data types, invalid ratings (outside 1–5), invalid timestamps (outside 2020-01-01...2025-12-31).

Each rule has a severity: error rows are rejected into data/errors/<table>_errors.csv with the reason warning rows are kept and repaired in preparation (e.g. missing age). Every flagged cell is logged in *_issues.csv.



3. Feature management

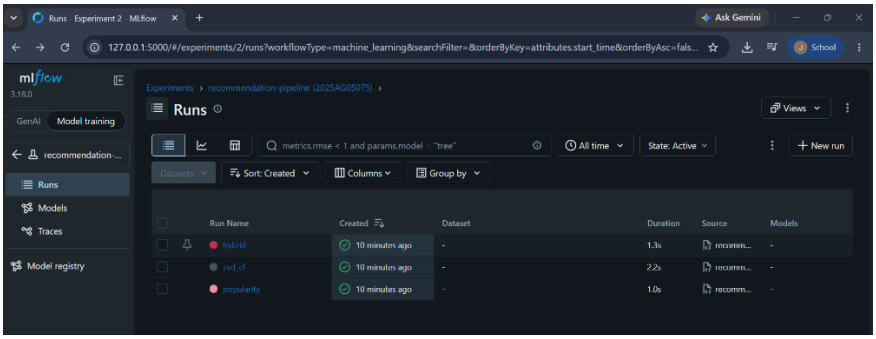
A file-based FeatureStore (Parquet offline store) builds and serves 19 documented features (full table: reports/feature_catalog.md): user – average rating, rating std, activity count, positive ratio, days since last interaction, tenure, preferred category, age; item – popularity, popularity percentile, raw and Bayesian-smoothed average rating, positive ratio, age, recency, number of genres; interaction – user–genre affinity, preferred-category match, expected-rating baseline. All features are computed from the training split only, as of the last training timestamp, and are persisted once so they can be reloaded without recomputation. The hybrid model consumes the genre-affinity feature directly from the store (via a vectorised version of user_genre_affinity), while the popularity and SVD models learn from the interaction matrix. The store exposes get_user_features, get_item_features and get_pair_features as a single read interface for prediction-time lookups.

Custom layer chosen over Feast to keep setup to pip install.

4. Models and tracking

- **Popularity (baseline):** ranks all items by training interaction count, rating prediction = Bayesian-damped item mean.
- **SVD collaborative filtering (personalised):** truncated-SVD matrix factorisation (40 factors, scikit-learn). Ranking head = PureSVD on the binary interaction matrix, rating head = user-mean baseline + SVD on the mean-centred rating matrix (for RMSE/MAE).
- **Hybrid:** per-user min-max-scaled blend 0.6·CF + 0.3·genre affinity (feature store) + 0.1·popularity.

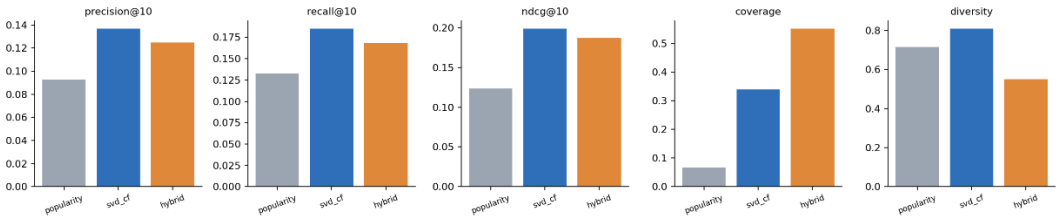
Each model is an MLflow run (SQLite backend) logging parameters, seed, training time, the pickled model and the feature catalog evaluation metrics are logged back to the same runs, so one run holds everything needed to reproduce and compare a model.



5. Evaluation and results

For each test user with at least one liked test item, already-seen training items are excluded and the top K = 10 are scored. Precision/Recall/NDCG@10 follow their standard definitions, Coverage = share of the catalogue appearing in any user's list, Diversity = mean intra-list distance (1 – average pairwise cosine similarity of genre vectors). RMSE/MAE are computed over all test ratings.

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
Popularity	0.0927	0.1329	0.1234	0.0662	0.7146	0.8466	0.6910
SVD-CF	0.1369	0.1853	0.1988	0.3400	0.8101	0.8495	0.6820
Hybrid	0.1246	0.1683	0.1867	0.5525	0.5512	0.8495	0.6820



Findings:

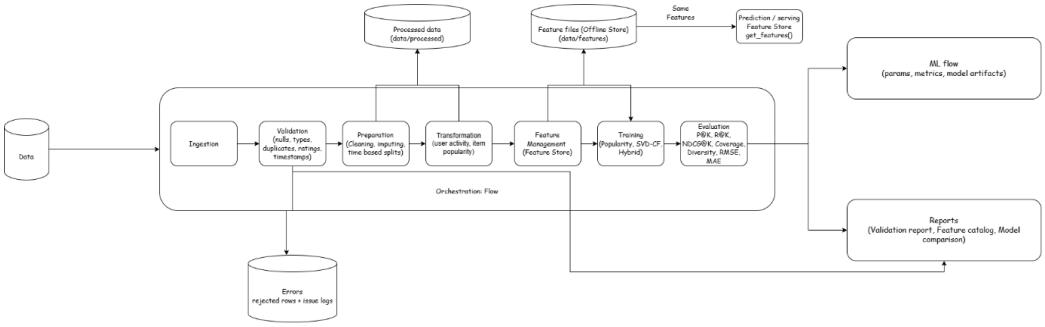
The personalised SVD model beats the baseline on every ranking metric (NDCG@10 +61%, Precision@10 +48%) and reaches 5× the catalogue coverage, showing that the validated, time-

split data and features carry real personal signal. The hybrid trades ~6% NDCG for the highest coverage (55%) but lower genre diversity, because the genre-affinity term pulls each list towards the user's favourite genres. On rating prediction the models are essentially tied (RMSE 0.847 vs 0.849, SVD wins on MAE): ratings in the synthetic data are dominated by user/item biases plus noise, which the damped item mean already captures. The blend weights were set a priori and not tuned on the test set.

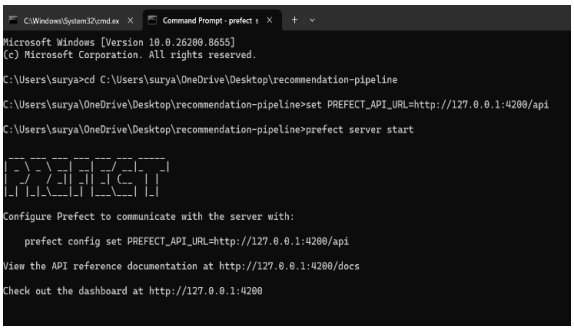
6. Orchestration and reproducibility

Prefect flow of eight tasks defined in dags/recommendation_flow.py (ingest, validate, prepare, transform, features, train, evaluate, report). Each task receives its upstream result, so Prefect derives the dependency graph; tasks retry once on failure; --serve deploys it on a daily cron. A successful run (8 task runs, ~8 s) is shown in results/prefect_serve_run_cmd.png, results/prefect_successful_run.png . A plain run_pipeline.py executes the same stage functions without an orchestrator, and 5 pytest tests cover validation rules, the time split and the metrics.

Architecture flow



Orchestration





7. Limitations

- Single time-based split and a single seed – no cross-validation or confidence intervals, hyper-parameters (factors, blend weights) were not tuned.
- Cold-start is not handled: test users/items unseen in training receive popularity or global-mean fallbacks only.
- File-based feature store has no online low-latency serving, point-in-time joins or versioning (Feast would add these); batch re-computation only.
- Evaluation counts only explicit likes (rating >= 4) as relevant and ignores position bias; diversity is genre-based only.
- Future work: implicit-ALS / LightFM, hyper-parameter search tracked in MLflow, data-drift checks inside the DAG, incremental feature updates